

Benchmark Report: nbody

Five bodies, twenty thousand steps, and an interpreter between every calculation

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

A probe is falling toward Neptune, and somebody has to know where it will be in six hours.

Predicting motion means repeatedly adding up gravitational forces. This benchmark captures that computational core in miniature: the Sun and four gas giants, ten interacting pairs, and twenty thousand steps. The physics is compact; in Python, the machinery around each calculation is not. Our optimization asks how much of that machinery we can remove.

1. Workload and result

The Computer Language Benchmarks Game's nbody uses symplectic Euler integration under Newtonian gravity. Each timed iteration evaluates energy, calls `advance(0.01, 20000)`, then evaluates energy again. The stock kernel uses only the standard library, with no I/O in the timed region. State consists of five position/velocity/mass records (35 float values); the ten body pairs are constructed once and reused.

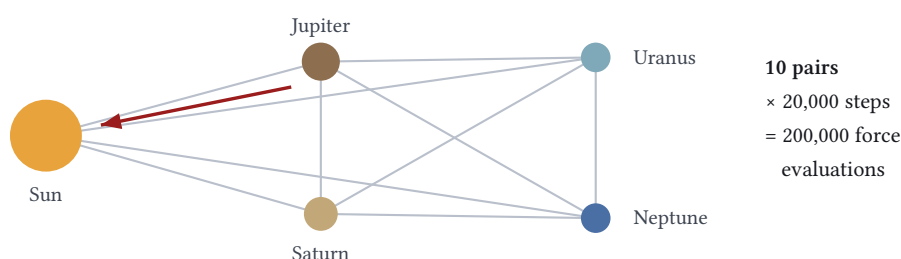


Figure 1: Ten fixed pairs × 20,000 steps = 200,000 pair-force evaluations.

Configuration	Mean ± SD	vs stock
Stock Python	231.23 ± 3.40 ms	1.00×
Optimized Python	141.28 ± 4.27 ms	1.64×

The optimization reduces runtime by **38.9%**, exceeding the course's 7% requirement. It specializes the interaction schedule into local-variable arithmetic while preserving operation order. Final state and energy compare exactly equal to stock after 20,000 steps. Energy is a useful physical sanity check; componentwise state comparison is the stronger equivalence check.

Measurement scope. Course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12, pyperformance 1.14.0; 120 measured values per configuration. Source: `results/baseline_nbody.json` and `optimized_nbody.json`. SD denotes sample standard deviation, not a confidence interval. Profiling is separate from timing; the shared appendix records methods and provenance.

2. Profiling and optimization

Python-frame sampling places almost all benchmark work in `advance()`. The C-level debug-build profile identifies list access, generic arithmetic dispatch and float allocation as substantial costs. Its list-access symbols sum to about 14.4% of sampled self time across coordinate indexing, loads and stores. These debug-build percentages locate costs to investigate; release-build timing establishes the optimization's benefit.

Full recorded flame graph

Highlighted regions - enlarged with surrounding frames

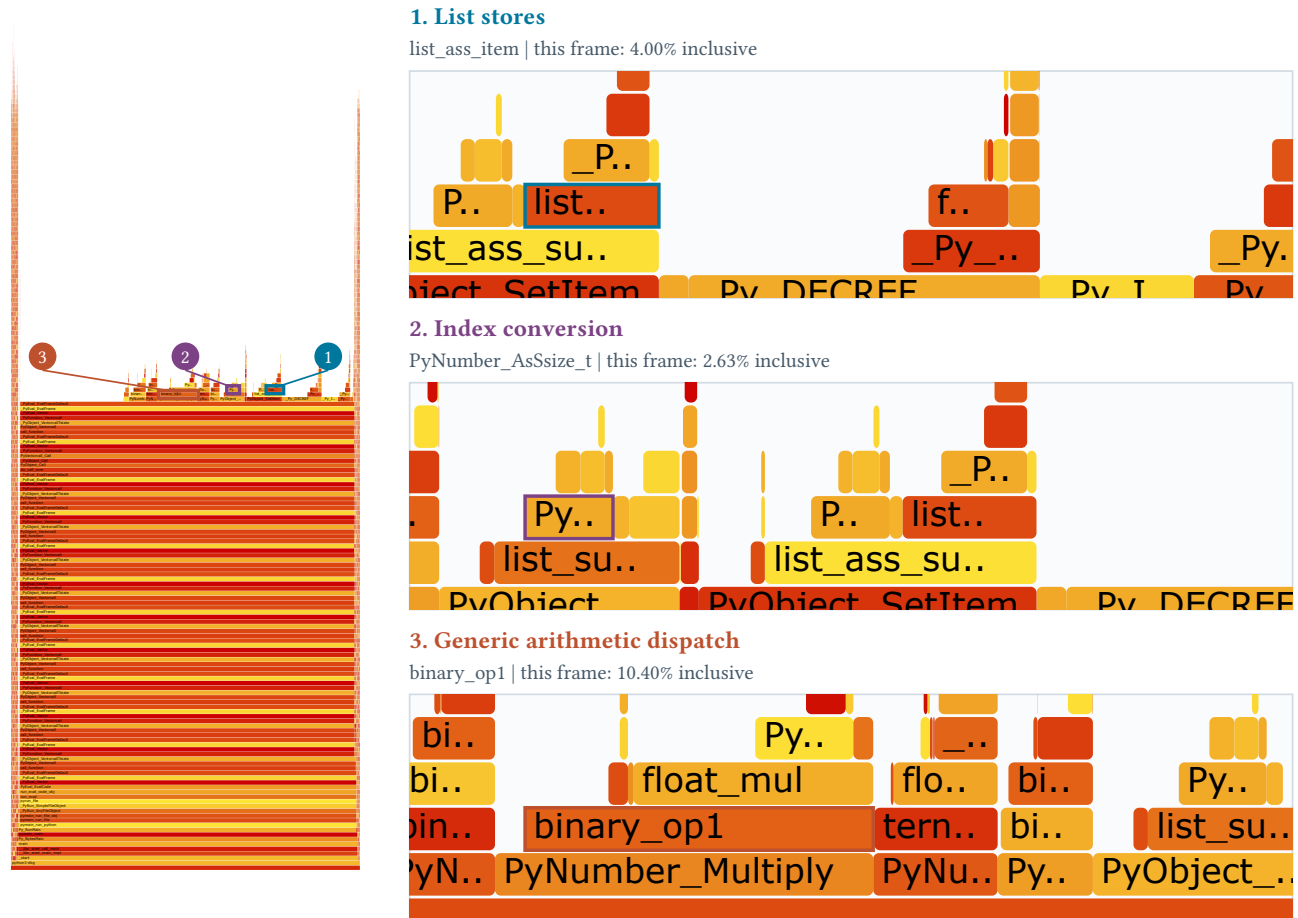


Figure 2: Full stock C-frame profile (debug CPython), with the same call paths outlined and enlarged. All original frames, widths and startup context are retained. Percentages describe the selected inclusive call path, not a function's total self time.

Specialize the fixed schedule

The original loop revisits the same pairs and masses on every step. At import time, the optimized version emits an `advance()` function with the pair loop unrolled, coordinates in Python locals, and masses as literals. It loads state before the step loop and writes it back afterwards, so energy reporting still observes the original body objects.

This removes list subscripting from the generated step loop and avoids repeated pair unpacking. List operations remain at function entry and exit, amortized over 20,000 steps.

Correctness constrains the arithmetic. The kernel retains $dt * (dsq ** -1.5)$ and the stock pair-update order. A square-root/division replacement changes rounding and is not used in the exact software tier. The emitter accepts a body list and pair schedule, so the same transformation extends to larger systems.

3. Native execution: less work, not higher IPC

The Rust/PyO3 kernel stores state in a `System` object and executes all 20,000 steps through one `advance(dt, n)` call; both energy evaluations also run in Rust. On the tested VM build, state and energy also compare exactly equal to stock. This is evidence for the tested compiler, library and workload, not a portability guarantee for every floating-point build.

Same benchmark file	Mean $\pm$ SD	vs Python
Python backend	141.08 $\pm$ 1.91 ms	1.00×
Native backend	9.493 $\pm$ 0.047 ms	14.86×

These are 120-value VM measurements from `fallback_nbody.json` and `native_nbody.json`. The separate kernel experiment (228.49 $\rightarrow$ 9.48 ms, 24.1 $\times$) compares native execution with **stock** Python. The table compares it with the already optimized Python implementation. Both the baseline and timing protocol must be specified when comparing these ratios.

Backend selection is explicit: `HWSW_BACKEND=python|native|auto`. Native mode fails if the wheel is unavailable; auto mode falls back to Python. Pyperf workers must receive the variable through `--inherit-environ HWSW_BACKEND`; result metadata identifies what ran.

Matched-work CPU counters

A separate warm-loop experiment repeats the same timed work 64 times per run on a fixed guest CPU. The table reports medians of three runs, normalized to one 20,000-step iteration. Counters exclude initialization and warmup; Appendix A2 documents gating and event validation.

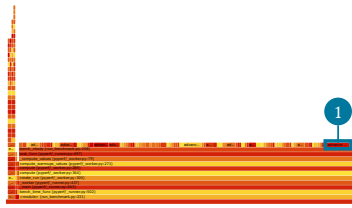
Metric per iteration	Python	Native
Elapsed time	140.29 ms	9.473 ms
Instructions	1,103.62 M	41.32 M
Cycles	334.56 M	22.56 M
IPC	3.30	1.83
Branches	185.97 M	3.92 M
Branch misses (rate)	585,719 (0.315%)	61 (0.00156%)
Generic cache references	2,980	279
Generic cache misses (rate)	37 (1.26%)	12 (4.00%)

The main gain is 26.7 $\times$ fewer instructions and 14.8 $\times$ fewer cycles. Compiled arithmetic avoids Python object handling and dispatch, removing most branches too. IPC actually falls: native execution retires fewer instructions per cycle, but needs far fewer instructions to finish. Higher IPC is not synonymous with a faster program.

Generic cache misses are sparse in both runs. Native has fewer absolute misses even though its miss/reference ratio is higher; the denominator shrinks more. These small counts do not establish cache latency as the bottleneck. L1-load counts were invalid and are not reported.

4. What changes when N grows?

Full recorded flame graph



Highlighted regions - enlarged with surrounding frames

1. Generated integration loop

advance | this frame: 6.42% inclusive

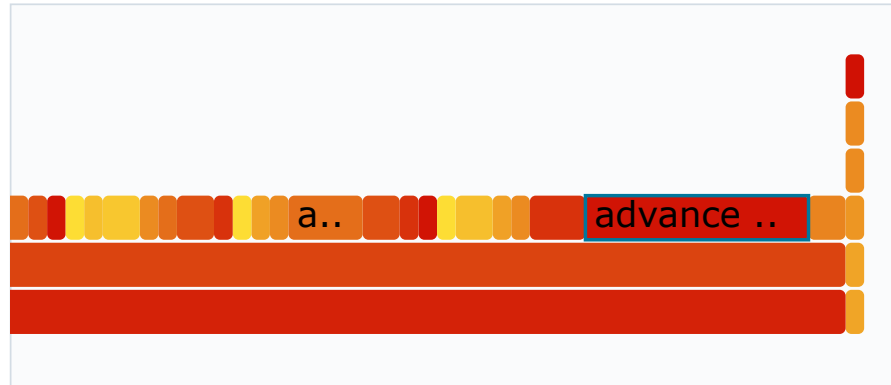


Figure 3: Full optimized Python-frame profile with the integration call path enlarged. Integration remains the hotspot.

Independent normalization does not display the absolute runtime reduction from 231 to 141 ms.

At five bodies, Barnes-Hut is **3.92× slower** than direct force evaluation on the VM. Tree construction and traversal do not pay off for ten pairs. Development experiments also found that struct-of-arrays and NumPy lose at this size (dev/nbody/FINDINGS.md). FMM was not benchmarked, so no measured FMM speedup or lower bound is claimed.

The `--bodies N` extension adds deterministic outer orbits while preserving the five-body default. Source generation grows as $O(N^2)$, increasing compilation time and memory use. The shipped implementation falls back to a rolled loop above 20,000 pairs. Detailed unrolling and local-slot experiments used **WSL2 / CPython 3.10.21**; their timings and memory limits are development observations, not VM results.

The VM force-evaluation sweep below includes a rebuilt Barnes-Hut tree at each evaluation ($\theta = 0.5$, best of seven). It evaluates an extended workload, not the required $N = 5$ benchmark.

N	Direct	BH	BH/direct	Median rel. error
5	0.020 ms	0.077 ms	3.92×	0
100	4.746 ms	8.410 ms	1.77×	1.6e-5
300	42.977 ms	42.489 ms	0.99×	3.3e-5
800	313.489 ms	158.846 ms	0.51×	8.7e-5
1,600	1,254.660 ms	382.471 ms	0.30×	1.8e-4

The crossover is near $N = 300$ for this distribution. Median relative acceleration error is not a worst-case or trajectory-error bound. The separate VM native sweep gives 21–22× over rolled Python from $N = 100$ to 3,200. Step counts approximately equalize pair-updates, with integer rounding increasing work at the final row. Both sweeps are in `results/bigN_sweep.txt`.

5. Hardware acceleration: current design

`grape_pipeline` retains state on the device and executes a complete advance(dt, n) request. Force computation suits a scheduled datapath, while velocity accumulation must respect dependencies between pairs sharing a body. Steps remain serial.

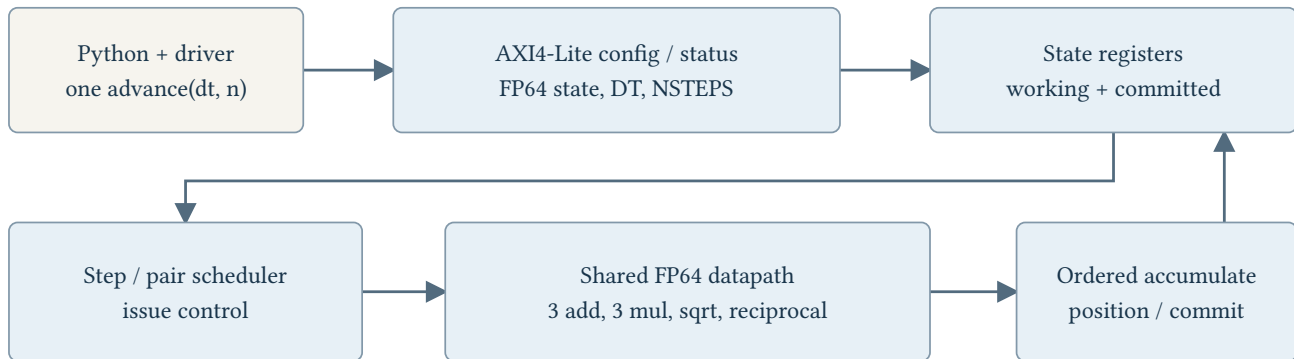


Figure 4: Current datapath and software boundary, summarized from the approved architecture. Multiplication and accumulation round separately; there is no FMA unit.

Datapath. Three FP64 add/subtract units, three multipliers, one square-root unit and one reciprocal unit share a scheduled 290-operation step. The square root uses radix-4 iteration; the reciprocal uses a ROM seed and refined fixed-point arithmetic. An ordered accumulation sequencer resolves body-component dependencies before positions are updated and committed.

Interface. A 32-bit AXI4-Lite slave exposes FP64 body state and DT, 32-bit NSTEPS, pair configuration, control/status, counters and IRQ. Software loads state and parameters, starts one request, waits, then reads state. There is no per-step DMA or Python call. The Rust object demonstrates this boundary; it is not a completed hardware driver.

Evidence / target	Current status
Step schedule model	123 cycles nominal; 127 at modeled latency corners
Clock target	50 MHz minimum; final timing/PPA not yet established
Conditional execution time	49.2–50.8 ms for 20,000 steps at 50 MHz, before interface overhead
RTL and verification	Synthesis and directed bring-up recorded; full coverage, sign-off and integration remain open

At that design point, modeled execution is about $4.6\times$ faster than stock Python and $2.8\times$ faster than optimized Python, but **slower than the 9.49 ms native implementation**. More units, a higher achieved clock, or reduced precision require measured area/timing trade-offs. No measured power advantage is claimed. The hardware’s square-root/reciprocal expression differs from stock `pow`. RTL is checked against its own arithmetic model. Comparison with stock targets relative energy error $\leq 1e-12$ and per-body position/velocity errors $\leq 2e-9 / 5e-11$ at completion. These tolerance targets are distinct from the software tier’s observed exact equality.

6. Conclusion

Specializing a fixed schedule cuts Python runtime by 38.9% while preserving the tested state and energy exactly. Native execution removes more interpreter work through the same coarse interface. The hardware design implements that boundary, but its benefit over native software depends on achieving a better latency/area point than the current model.

Evidence: `hw/grape_pipeline/docs/uarch.md`, `prd.md`, `testplan.md`, and `hw/PROGRESS.md` (status recorded 2026-09-05). Reproduction and supporting profiles: **report_appendix.pdf**. Repository: [HWSW_Project](#).